

RIPASSO - SETTIMANA I°

giovedì 23 febbraio 2017 21:40

-ECCESO 127:

in realtà
guardando
le cifre sarebbe
0



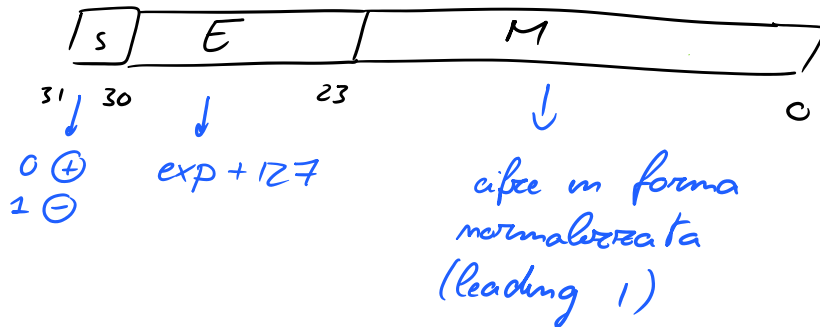
dato un E in IEEE754,
devo fare $E - 127$ per
avere il "vero"
esponente

$$\begin{aligned} -127 &\rightarrow 00000000 \\ -126 &\rightarrow 00000001 \\ &\vdots \end{aligned}$$

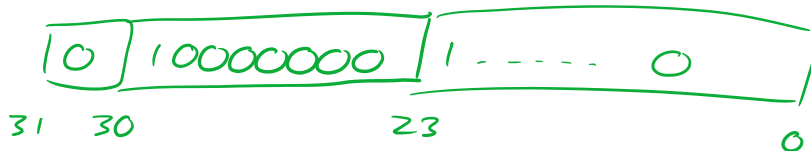
$$\begin{aligned} 0 &\rightarrow 01111111 \\ 1 &\rightarrow 10000000 \\ &\vdots \end{aligned}$$

$$+128 \rightarrow 11111111$$

IEEE 754



es:



$$0 \rightarrow +$$

$$10000000 \rightarrow \underbrace{128}_{\text{val. eff.}} - \underbrace{127}_{\text{bias}} = 1$$

$$1 \dots 0 \rightarrow 1, 1$$

$$\text{numero} = + 1, 1 \cdot 2^1 = + 11$$

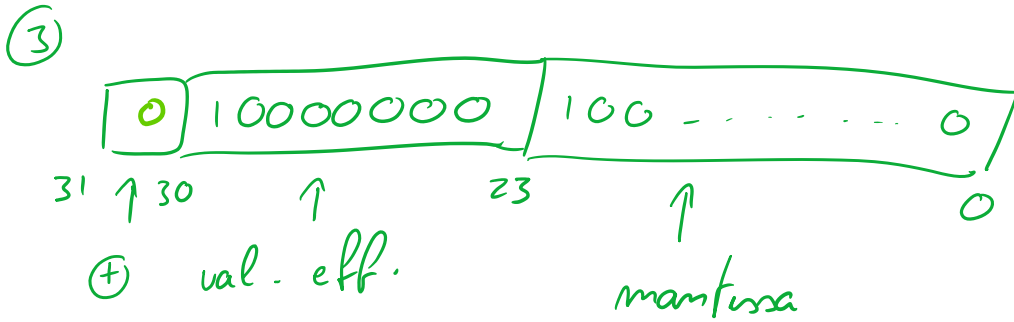
$1, 1 \cdot 2 = 11$ (la virgola si sposta di un posto a dx)

$$\text{es: } 7,7_{10} \cdot 10_{10}^1 = 77$$

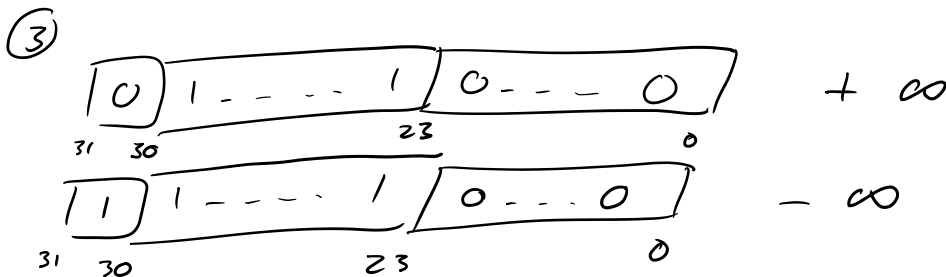
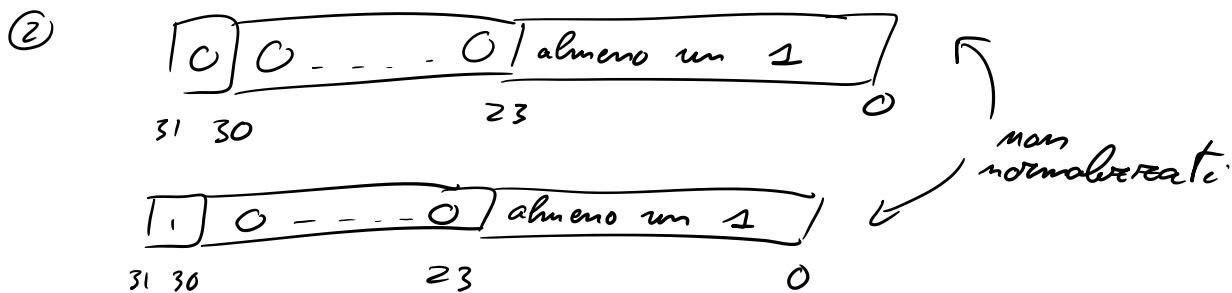
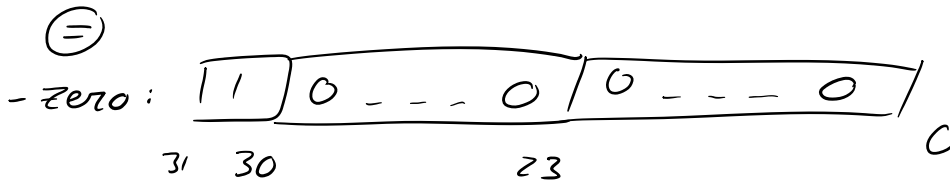
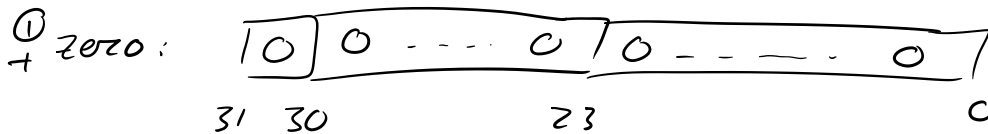
numero = $\oplus 11_2 = 3_{10}$

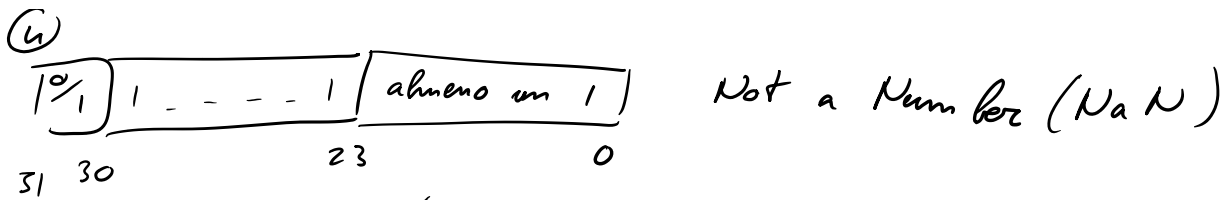
① forma normalizzata leading 1 : $1,1 \cdot 2^1 = 1,1000 \dots 0 \cdot 2^1$

② exp in excess-127 : $1 + 127 = 128$
 exp bias val. eff.



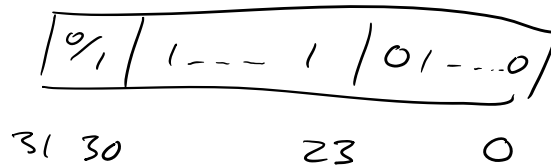
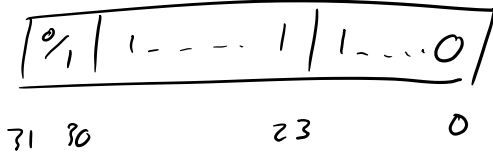
CONF. SPECIALI :





Q NaN
(Quiet)

S NaN
(Signaling)



undetermined operations

invalid operations

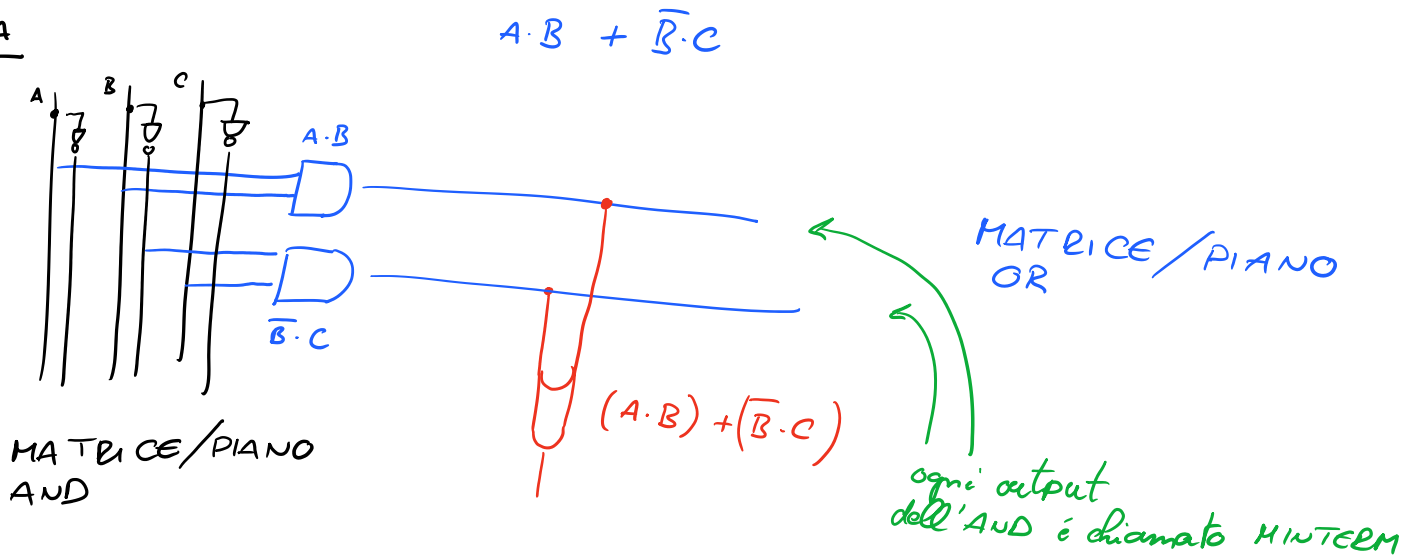
RIPASSO - SETTIMANA 2

sabato 4 marzo 2017 19:03

① Ogni funzione logica può essere implementata con le sole funzioni AND, OR e NOT

⇒ Ogni funzione logica può essere scritta in FORMA CANONICA con 2 livelli di porte logiche; la più importante F.C. è SUM OF PRODUCTS (realizzabile fisicamente con un PLA)

PLA



VANTAGGI:

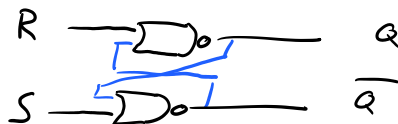
- ① Se devo usare uno stesso prodotto più volte mi basta una sola porta logica
- ② Non ho bisogno di porte logiche per configurazioni tutte false

ROM: come PLA ma PIANO AND NON programmabile (vengono calcolati tutti i MINTERMS). In pratica mi salvo la tabella di verità e visto che sono saldature fisiche i dati vengono salvati anche dopo lo spegnimento.

RIPASSO - SETTIMANA 3

domenica 12 marzo 2017 15:33

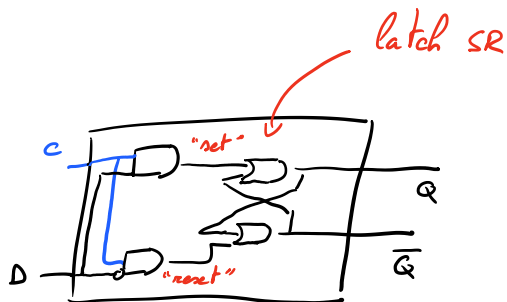
LATCH SR
(asimmetrico)



S	R	Q	Q̄
0	0	memorizza	
0	1	0	1
1	0	1	0
1	1	non ammesso	



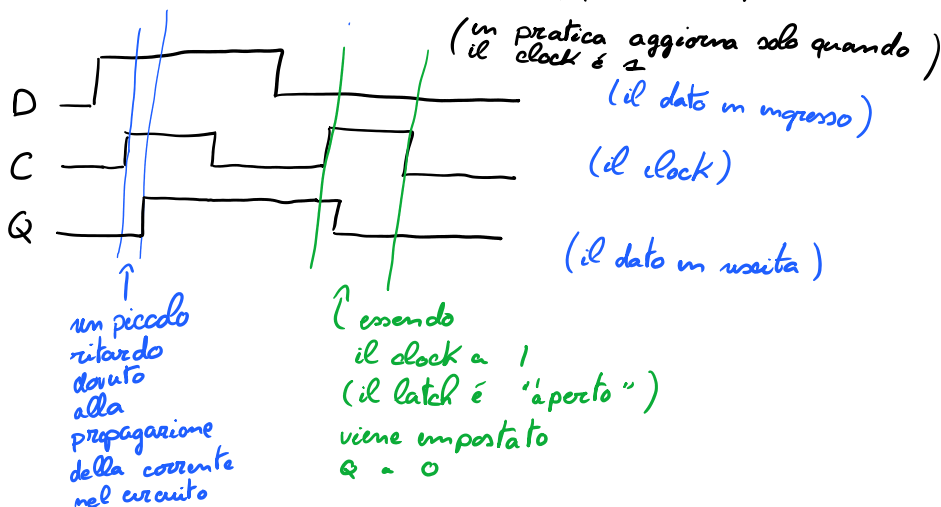
LATCH D
(simmetrico, aggiorna durante clock)



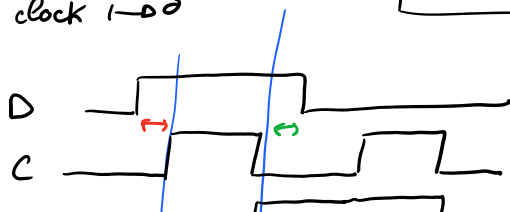
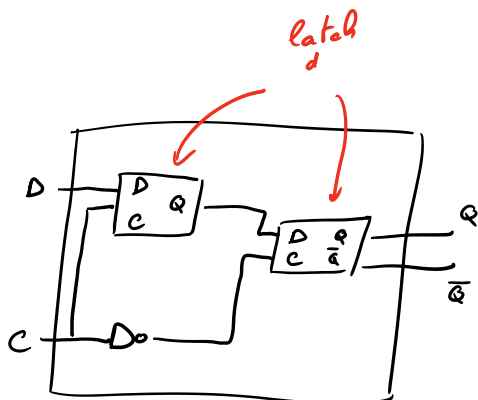
① evita la situazione
"

⊕

C	D	S	R	Q	Q̄
0	0	0	0	stato precedente	
0	1	0	0	0	1
1	0	0	1	1	0
1	1	1	0	1	0

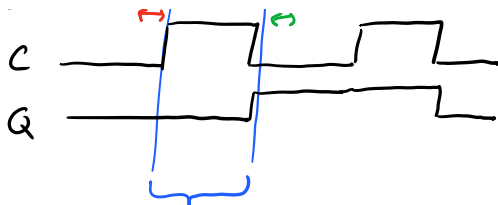


FUP FLOP D
(come latch D ma si aggiorna con FALLING EDGE, cioè quando clock 1→0)



↔ = SETUP TIME

tempo minimo per cui il dato deve essere valido prima del falling edge



D e C sono già ad 1, ma diversamente da latch d l'aggiornamento avviene solo quando C passa da 1 a 0

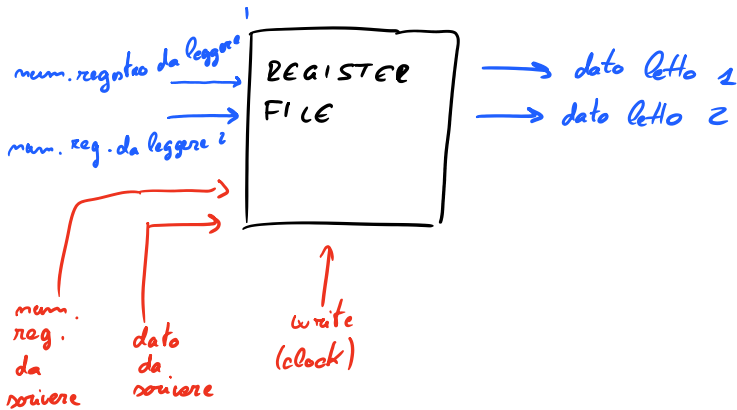
tempo minimo per cui il dato deve essere valido prima del falling edge

← = HOLD TIME

tempo minimo per cui il dato deve essere valido dopo il falling edge

REGISTER FILE

= insieme di registri che possono essere letti/scritti; fondamentale per il DATAPATH (= operazioni aritmetiche)



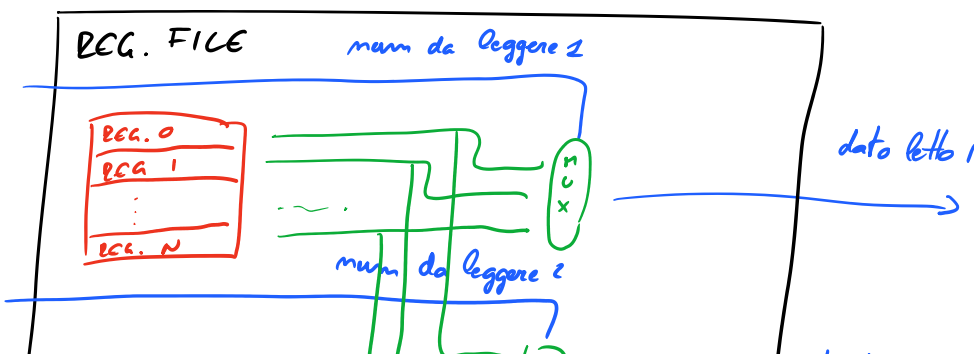
} visto che leggere un registro non ne modifica lo stato occorre solo fornire in ingresso il num. del registro mentre in uscita si avrà il dato

per scrivere mi serve:

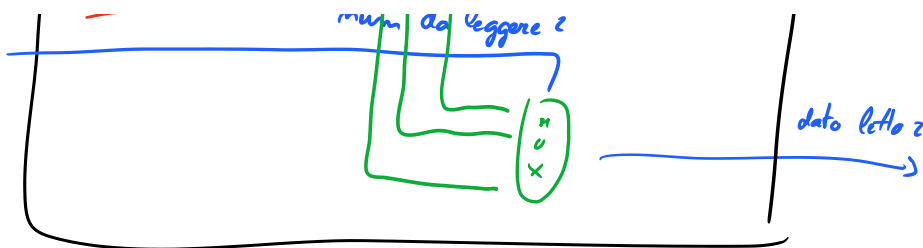
- ① sapere in quale registro devo scrivere
- ② sapere che dato devo scrivere
- ③ il clock (dopotutto i registri sono flip-flop D)

↑ ovviamente non avendo il clock succederebbero dei casini (ad es.: lettura di uno stato inconsistente)

come viene implementata la lettura?

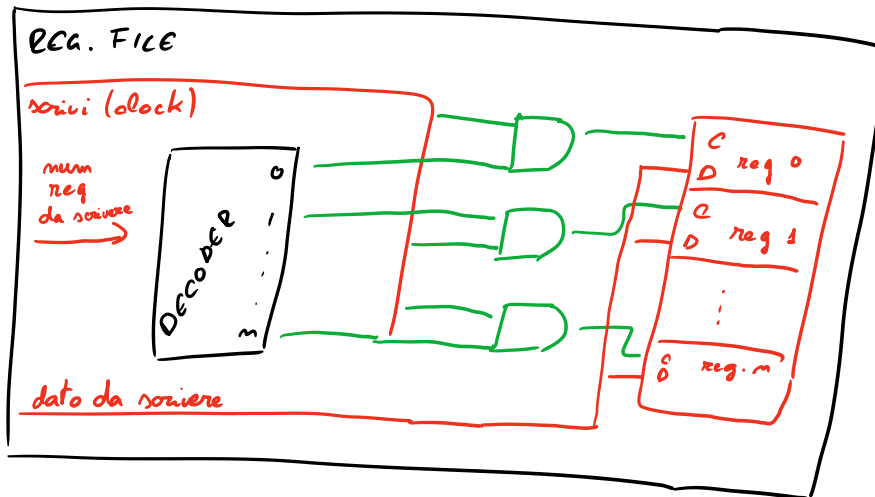


In pratica ricevo in ingresso il numero del registro da leggere, collego questo numero ad un multiplexer che mi manda in uscita solo il dato desiderato



twaker che mi
manda in uscita solo
il dato desiderato

come viene implementata la scrittura?

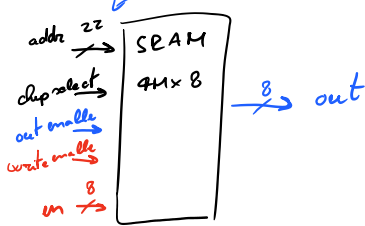


In pratica:

- ① grazie al decoder, ricevo in ingresso il num del registro da leggere e attivo solo il filo corrispondente
- ② grazie all'and posso aspettare fino a quando il clock non scende (falling edge)
- ③ quando il clock è in falling edge l'and diventa vero, quindi l'ingresso C del registro (flip-flop d) si attiva e posso scrivere all'interno.

RIPASSO - SRAM/DRAM/FSM

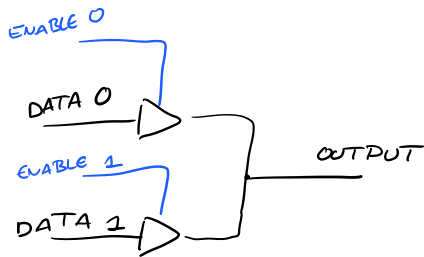
SRAM



$U = 2^2$ $M = 2^{20}$ $UM = 2^{22}$ locazioni indirizzabili; ciascuna di 8 bit (quest'ultima non mi)
 $\leftarrow$ alterza
 $\leftarrow$ lunghezza
 $\leftarrow$ interezza x addr

- CHIP SELECT deve essere attivo sia x lettura che x scrittura
- ADDRESS contiene l'indirizzo da cui leggere / su cui scrivere
- per leggere devo attivare out enable; il dato ottenuto sarà in 'out'
- per scrivere devo attivare write enable e fornire il dato da scrivere in 'in'

come viene realizzata la lettura?



- Vengono utilizzati dei THREE STATE BUFFER

(usare un MUX sarebbe impossibile) in quanto troppo grande!

sono implementati all'interno dei flip-flop.

3-STATE BUFFER

X	C	Z
0	0	ALTA IMPEDENZA
1	0	ALTA IMPEDENZA
0	1	0
1	1	1

non passa nulla!



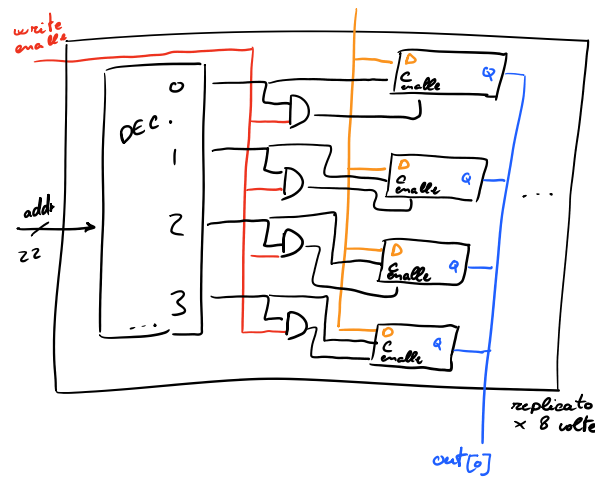
N.B.: ALTA IMPEDENZA

$\neq 0$

nel primo caso il componente è "spento", nel secondo fa passare una piccola tensione (val. logico 0)

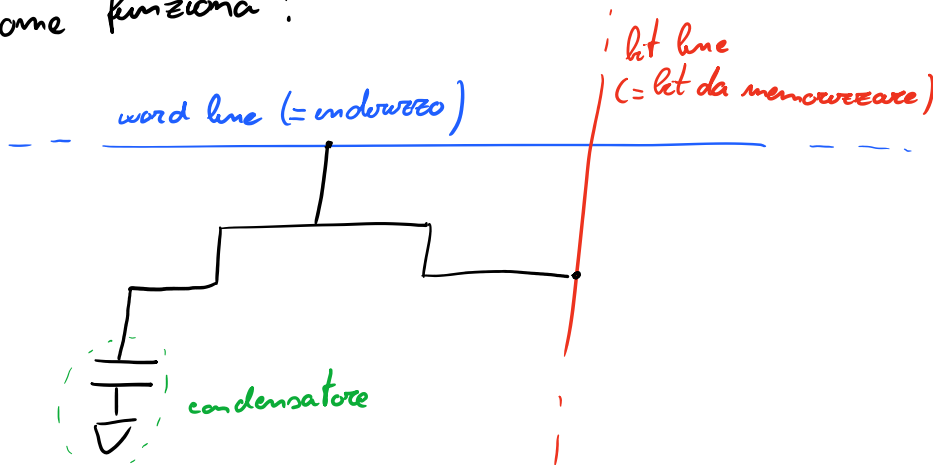
come viene realizzata la scrittura/lettura?

data[0]



DRAM

- al contrario delle SRAM che usano porte invertenti, le DRAM usano dei transistori (il dato deve essere "rinfrescato"/riscritto ad ogni ciclo di clock, da qui il nome "dynamic")
 - DRAM 5/10 volte più lente di SRAM; in compenso è più facile da realizzare in quanto ogni cella usa un solo transistor
- come funziona?

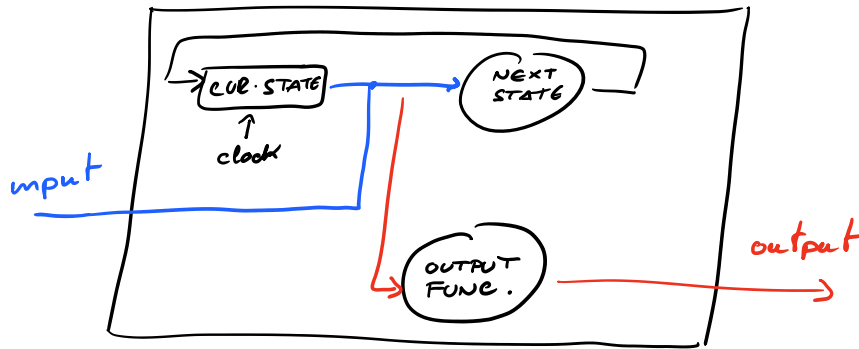


FSM

- modello che descrive CIRCUITI SEQUENZIALI (che hanno memoria)
- una FSM è composta da:
 - ① input e output
 - ② funzione next state (← N.B. è combinatoria, prende in ingresso l'input e lo stato e ritorna un nuovo stato)
 - ③ funzione output

- MACCHINA DI MOORE: (l'output dipende solo dallo stato)
 - ⊕ rapida REAGLY
 - usata come controller

macchina come controller



⑤ MACCHINA DI MEALY (l'output dipende da input + stato) • ④ piccola
< noi useremo solo Moore >

RIPASSO - INDIRIZZAMENTO

venerdì 7 aprile 2017 21:47

① REGISTER ADDRESSING

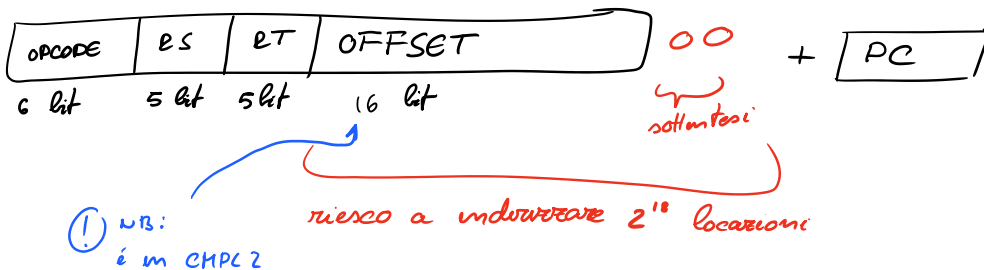
Metto l'indirizzo (a 32 bit) in un registro (a 32 bit).

Usato dall'istruzione j_r (nome registro). es: $j_r \$ra$

② PC-RELATIVE ADDRESSING ("caso particolare" di base addressing)

Usato da bne , beq , $bgtz$... ecc. Visto che statisticamente nei branch ("if") le istruzioni a cui devo andare sono vicine, posso usare il registro PC (Program Counter, contiene istr. successiva) a cui aggiungo un offset.

Inoltre visto che le istruzioni vanno allineate a indirizzi multipli di 4, posso sottintendere i due bit a destra che saranno sicuramente 0. es: $beq \$t2, \$t2, salto$

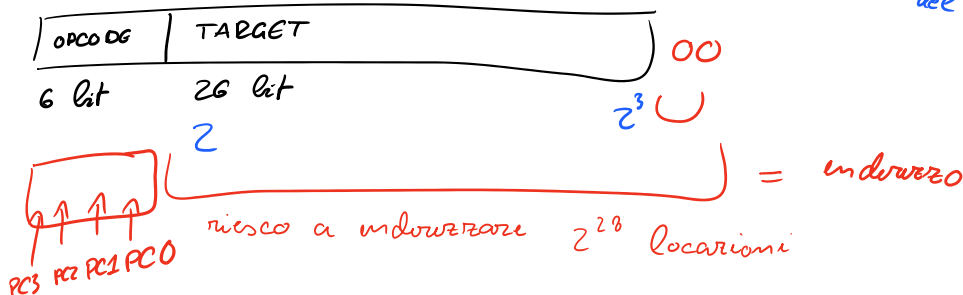


③ PSEUDO-DIRECT ADDRESSING

Utilizzato dall'istruzione j . Per gli ultimi due bit uso la considerazione del caso precedente. Inoltre metto in testa gli ultimi 4 bit del PC.

es: j prova

① Nel caso precedente SOMMO il PC, in questo AFFIANCO 4 bit del PC /



④ BASE ADDRESSING

Usato dalle istruzioni lw , sw , lb , sb , ...

In pratica fornisco un registro base (che contiene un indirizzo) a cui aggiungo un offset (numerico! NO registro). NB: mi permette di indirizzare byte (8 bit), halfword (16 bit) e word (32 bit).

es: lw \$t1, 0(\$t0), lb \$t3, 0(\$t2)

⑤ IMMEDIATE ADDRESSING

uso campo immediate (?)

CODICE SorgENTE (C, C++, Java)



COMPILATORE: traduce in ASM. OBJECT CODE



CODICE ASM



ASSEMBLER: traduce ASM in OBJECT FILE

problema: risolvere i riferimenti alle etichette locali.

L'assembler fa una prima passata e individua tutte le label. Ad una seconda passata risolve gli indirizzi. (Usa una SYMBOL TABLE, associa label - indirizzo). È necessario fare 2 passate in quanto è possibile che una label sia definita dopo l'utilizzo effettivo quindi è impossibile saperne a priori l'indirizzo.



OBJECT FILE: formato da:

- ① HEADER con informazioni relative alla grandezza e la posizione degli "altri pezzi" dell'object file

possono contenere riferimenti non validi che saranno risolti dal linker

- ② TEXT SEGMENT: contiene le istruzioni ASM in formato macchina.
- ③ DATA SEGMENT: contiene i dati da caricare in memoria in formato macchina
- ④ RELOCATION INFORMATION: contiene le posizioni assolute in memoria di istruzioni e dati. Può cambiare in futuro (fase di linking)
- ⑤ SYMBOL TABLE
- ⑥ INFORMAZIONI DI DEBUGGING



LINKER: traduce OBJECT FILE in ESEQUIBILE (.exe) (anche ④ di 1) (2 solo!)

problema: se un programma è costituito da tante file diversi, ricompilarli tutti sarebbe uno spreco di tempo. Inoltre quando scriviamo un programma utilizziamo delle procedure / "metodi" proprietari del linguaggio di programmazione, che devono quindi essere inclusi nell'eseguibile. (es: System.out.print())

Il linker, quindi: ① cerca le procedure "metodi" in n

nessa eseguire. (es: `system.out.print()`)

- Il linker, quindi:
- ① cerca le procedure "native" del linguaggio di programmazione e le accoda all'eseguibile
 - ② risolve i riferimenti tra file diversi (etichette globali)
 - ③ modificando le "RELOCATION INFORMATION" determina le posizioni in memoria di dati e istruzioni



ESEGUIBILE (.exe)

A questo punto posso eseguire l'eseguibile quante volte voglio. Ogni volta che eseguo il file utilizzo la parte del S.O. (Sistema Operativo) chiamato **LOADER**.

ESEGUIBILE (.exe)



LOADER: carica il programma in memoria ("si avvia il programma")

- ① Legge l'header del file .exe per capire quanta memoria allocare (per istruzioni e variabili)
- ② Alloca la memoria
- ③ Carica istruzioni e dati in memoria
- ④ Copia gli argomenti passati al main (args in Java)
- ⑤ Inizializza i registri
- ⑥ Chiama il main

$$27_{10} \rightarrow 11011_2 \rightarrow \boxed{0}11011_{MS} \rightarrow \boxed{0}11011_{CMPL2}$$

$$\begin{array}{r|l} 27 & 2 \\ 13 & 2 \\ 6 & 2 \\ 3 & 2 \\ 1 & 2 \\ 1 & 2 \end{array} \begin{array}{l} 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{array} \uparrow$$

$$\begin{array}{l} 11011 \\ = 1 + 2 + 8 + 16 \\ = 27 \end{array}$$

$$-27_{10} \rightarrow \boxed{1}11011_{MS} \rightarrow$$

CMPL2

$$+ 27_{10} = \boxed{0}11011_{CMPL2} \rightarrow -27_{10} = 100101_{CMPL2}$$

$$\begin{array}{r} 01101 \\ 10010 \\ \hline 100101 \end{array}$$

$$\begin{array}{r} -27 \rightarrow 100101 \\ + 27 \rightarrow 011011 \\ \hline \end{array}$$

$$+ 27_{10} \rightarrow 00011011_2 \rightarrow 0x1B \rightarrow 33_{OCT}$$

$$\begin{array}{r} 011011 \\ \hline 33 \end{array}$$

$$0x2AF7$$

$$\rightarrow 0010101011110111_2$$

$$\rightarrow 1 \cdot 2^0 + 1 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 + 1 \cdot 2^5 + \dots$$

$$\begin{array}{c} 164_{oct} \rightarrow 001 \underline{110} \underline{100}_{BIN} \\ \text{red} \quad \text{blue} \quad \text{green} \\ 001 \quad 110 \quad 100 \end{array}$$

→ DEC

$$\underline{0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 + \dots}$$

$$+ 27_{10} = 11011$$

$$\boxed{0 \mid 10000011 \mid 1011 \dots 0}$$

→ norm: $1, \overbrace{1011} \cdot 2^4 = 11011$

→ exp 127: $4 + 127 = 131_{10} \rightarrow 10000011_2$

— $27, 5_{10} = 11011, 1$

$$\begin{array}{r} 0,5 \times \\ \underline{2 =} \\ 1,0 \times \\ \underline{2} \\ 0 \quad 0 \end{array}$$

→ norm: $1, 10111$
 $\cdot 2^4 = 11011, 1$

→ exp 127 → $4 + 127 = 131 \rightarrow 10000011_2$

$$\boxed{1 \mid 10000011 \mid 10111 \dots 0}$$

⊖ exp ecc. 127 = 131

vero exp: $131 - 127 = 4$

$1, 10111$

$$1, 10111 \cdot 2^4 = 11011, 1$$

$\underbrace{}_{27} \quad \underbrace{}_{1 \cdot 2^1}$

m 137_x

$$0, 137_x$$

$$2 =$$

$$-----, 5$$

$$-----, 1$$

$$\boxed{10} \mid 274_x$$

$$2 =$$

$$\boxed{10} \mid 548_x$$

$$2 =$$

$$\boxed{11} \mid 096$$

$$-----, 25$$

$$-----, 01$$

$$-----, 125$$

$$-----, 001$$

$$A = 0x41 \rightarrow 0100 \ 0001 = A$$

$$0100 \ 0001$$

$$\rightarrow 1 + 64 = 65$$

$$10_2 = 2$$

$$\begin{matrix} \swarrow \\ \text{all } 2 : 100_2 = 4 \\ \downarrow \\ \text{all } 2 : 1000_2 = 8 \end{matrix}$$